



DAY 21 / 55

The **else** Statement & **Debugging**

Optimize your conditional logic with if-else blocks and learn to find & fix bugs using Python's debugger.



The **else** Statement

Instead of writing **two separate if statements** to check opposing conditions, Python's `if-else` block lets you handle both cases in one structure — making your code cleaner and more efficient.

❌ **INEFFICIENT**

Two separate `if` checks — both are evaluated every time, even when unnecessary

✅ **EFFICIENT**

One `if-else` block — once the `if` is true, the `else` is skipped entirely

Condition



True? → if block



False? → else block



if-else in Action

Check if a number is even or odd using a single if-else block:

```
# Even or Odd checker  
num = 15  
  
if num % 2 == 0:  
    print("Even number")  
else:  
    print("Odd number")
```

Python

► Output

Odd number

num = 15



15 % 2 ≠ 0



else → "Odd"



What is Debugging?

Debugging is the process of identifying, analyzing, and removing "**bugs**" (errors) in your code. The term originates from early computing when a literal moth caused a computer malfunction!



Identify

Find where the problem occurs in your code by observing unexpected behavior or incorrect output.



Analyze

Understand *why* the bug happens by tracing through your code step by step using a debugger.



Fix

Correct the faulty logic or syntax error and verify the fix produces the expected results.



Types of Errors

● Syntax Errors

Typos, missing colons, or incorrect indentation that prevent code from running. Your editor highlights these immediately.

```
if num = 5 ← missing colon ":"  
    print("five")
```

● Logical Errors

Code runs without crashing but produces **wrong output**. These are harder to detect because no error message appears.

```
# Bug: Should multiply, not add  
area = length + width # ← wrong operator  
# Correct: area = length * width
```



Using the Debugger

VS Code and PyCharm both provide powerful built-in debuggers. Here are the key techniques:

1

Set Breakpoints

Click the left margin of the editor to place a red dot. Execution will **pause** at this line so you can inspect the program's state.

2

Start Debugging

Launch the debugger. The program runs until it hits your breakpoint, then pauses for inspection.

3

Step Over

Execute one line at a time. Watch variables change in real-time in the Variables panel as you step through.



Debugger in Action

```
1 num = 15
2 • if num % 2 == 0:
3     print("Even")
4 ► else:
5     print("Odd")
```

Variable Inspector

num	15
num % 2	1 (not 0 → else branch)



Key Takeaways



Use **if-else** instead of two separate if statements — it's cleaner and skips unnecessary checks.



Debugging = finding and fixing errors. Know the difference between syntax errors and logical errors.



Set **breakpoints** to pause execution and inspect variable values in real-time.



Use **Step Over (F10)** to execute line-by-line and trace your program's flow.



Next: More Conditional Logic

Day 22 — elif, Nested Conditions & More